Chapter route

How can multiple guests remain isolated while preserving instruction semantics, device interfaces, and resource bounds?

Prerequisites are OS structure, traps, address translation, and I/O.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Distinguish type 1 and type 2 hypervisors.
- Analyze trap-and-emulate and binary translation.
- Explain memory/I/O virtualization safety and costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

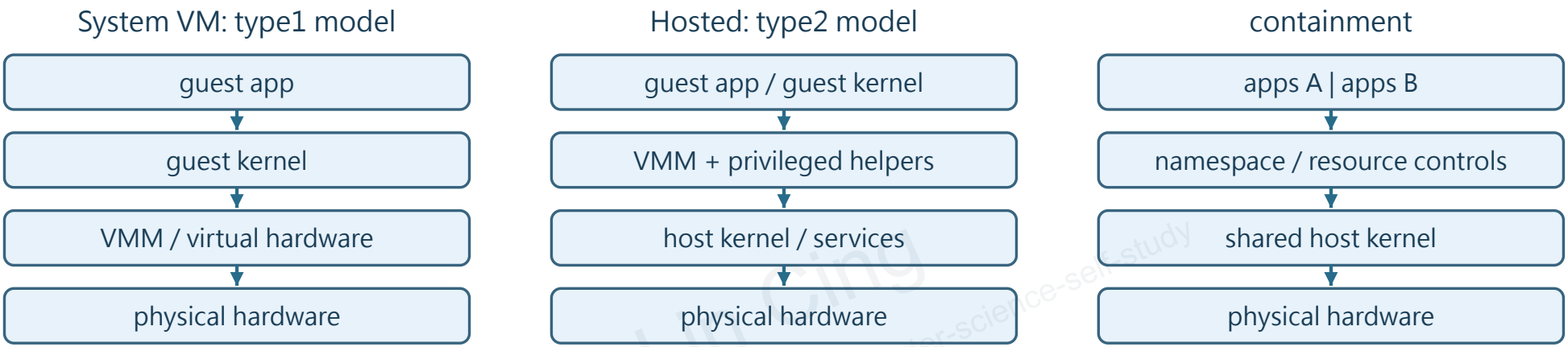
Overview

A system VM provides CPU, memory, device interfaces to a guest OS. A vCPU stores state; a pCPU executes it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Host, guest, and VMM boundaries

Resource-control layers differ: VMs and shared-kernel containers



The VMM/hypervisor manages physical resources and implements virtual interfaces. Guest applications call their guest OS, which controls virtual CPUs/devices. A guest syscall need not immediately enter the host kernel.

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

Host, guest, and VMM boundaries (continued)

A vCPU retains guest registers, PC, state, and virtual interrupts until scheduled on a pCPU. Two VMs can both name physical page5 while second-stage mappings select different host frames. The VMM isolates shared CPUs, memory, devices, and management interfaces.

System VMs, process VMs such as JVM, cross-ISA emulators, and OS containment expose different abstractions. Identify the provided interface, kernel ownership, and physical-resource controller rather than relying on a type number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

History

VM/370 minidisks partition shared resources. Fidelity, resource control, and efficiency are separate goals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Motivation and design goals

IBM VM/370 in the 1970s presented separate physical-disk regions as guest minidisks for multiple OSs on one mainframe. The abstraction preserves ranges/isolation rather than exposing unrestricted disk registers.

Fidelity preserves specified guest-visible behavior except explicit differences. Resource control prevents guests seizing VMM-controlled resources. Efficiency permits most safe work to execute directly. Neither goal promises a fixed slowdown or identical timing for every workload.

Reliable interception of sensitive instructions needing mediation is a common sufficient condition for traditional trap-and-emulate. Otherwise use translation, paravirtualization, or hardware assistance. Full software emulation can interpret an ISA without dedicated VM instructions; hardware changes technique/efficiency, not the logical possibility of emulation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Benefits and features

Consolidation still faces simultaneous peaks and shared failures. Snapshots, clones, and backups have different consistency/failure boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Isolation and shared failure

VMs support OS development, multiversion testing, and low-utilization server consolidation. Hypervisor/device-emulator vulnerabilities or a shared failed host can still affect several VMs. CPU, memory, I/O quotas are needed against interference; separate addresses alone do not prevent slow neighbors.

An eight-core host reserves one for VMM. A/B/C peak at 3/3/2 cores, totaling8>available7. Average demand of one each cannot guarantee no queueing at simultaneous peaks. Overcommit depends on workloads/service goals and creates no compute capacity.

Shared networks/disks introduce trust boundaries. Management can clone, snapshot, read disk images, making its authority part of the TCB even when guests are isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Suspend, snapshots, clones, and backups

Suspend retains resumable CPU/RAM/device state; a disk snapshot retains block versions at a point; a clone creates an independently runnable copy. Copying active disk files need not capture one consistent instant or matching RAM/disk state. Use shutdown, supported consistent snapshots, or guest application quiescence.

Crash consistency resembles recoverable power-loss disk state; application consistency additionally coordinates flush/transactions. External mail, payments, databases outside the snapshot are not undone by rollback. Clones must address duplicate IP/MAC, machine identities, and credentials.

Snapshots sharing the source's failure domain are not independent backups. Templates replicate both configurations and embedded vulnerabilities/credentials. Cloud/VDI APIs retain these isolation/external-state constraints.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

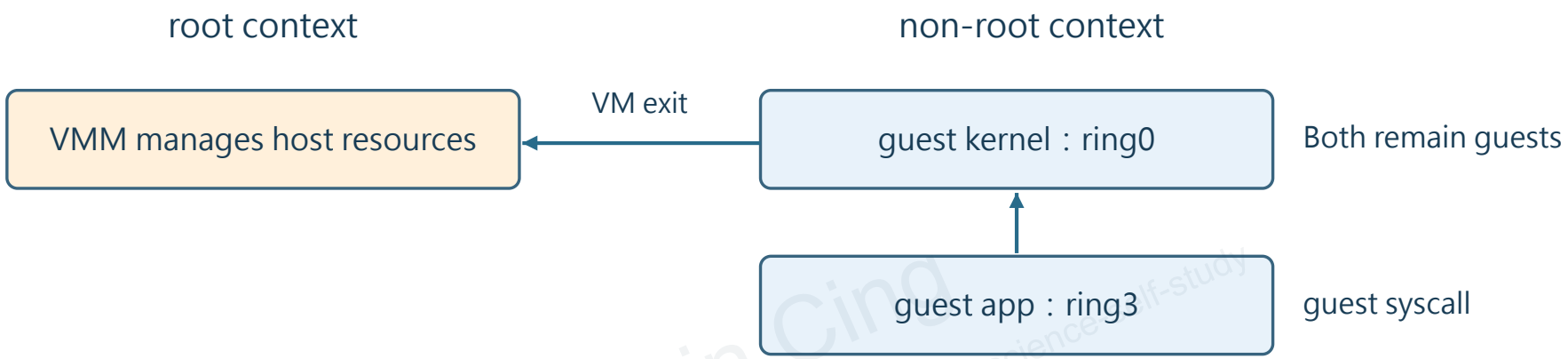
Building blocks

Traps, translation, hypercalls, and hardware assistance have different premises. Shadow and two-stage translation require distinct mapping/permission/cost arguments.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Virtual modes versus physical modes

User/kernel and root/non-root are separate privilege axes



Hardware-assisted model: non-root ring0 ≠ host root. Selected events exit; not every guest syscall.

In a traditional model without a separate VM mode, the VMM holds physical privilege while guest kernel/user execute with reduced physical rights. VMM retains virtual guest mode. A guest-kernel privileged operation traps, is checked against virtual authority, and is emulated. An illegal guest-user operation must produce a guest exception rather than gain authority by entering VMM.

Virtual modes versus physical modes (continued)

When a guest kernel disables interrupts, VMM clears vCPU.virtualIF and delays that guest's maskable interrupt injection. It must not permanently disable host interrupts serving other guests/devices. Re-enabling permits pending events according to the virtual architecture while retaining host control.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sensitive instructions that do not trap

An instruction may silently ignore some changes at reduced physical privilege, leaving a guest kernel's expected flags wrong without notifying VMM. Historical x86 POPF behavior under particular flags/privileges illustrates the gap. Lack of trap does not imply harmlessness.

Binary translation reads guest blocks, replaces mediated operations with safe equivalent VMM behavior, and caches executable translations. It must preserve indirect branches, self-modifying code, page-permission changes, and exception-visible state. Changed source code invalidates stale translations.

Paravirtualization instead changes a cooperative guest to use hypercalls. It can address similar problems with different guest ABI/modification requirements. Hardware assistance does not remove scheduling, device-emulation, and I/O costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hardware-assisted execution

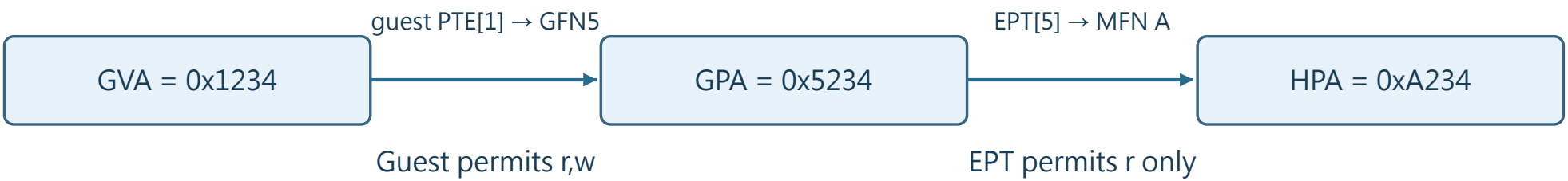
Root/non-root or host/guest context adds an axis beyond user/kernel. Guest kernel can run non-root ring 0, app non-root ring 3, while VMM manages root context. VM entry loads guest state; configured events exit, save state/provide reason, and let VMM handle/resume.

Architecture/controls determine exits; not every instruction or syscall exits. Fidelity/control require correct intercepts, state retention, event injection. IOMMU and interrupt remapping isolate/route device activity separately from CPU virtualization. EPT alone does not constrain all DMA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shadow tables and hardware two-stage translation

Compose addresses and permissions at both stages



4096 B pages preserve offset 0x234 at both stages. Read passes; second stage denies write.

Shadow maps VPN1 → MFN A directly, requiring synchronized guest PTE updates and invalidation.

Guest tables map GVA→GPA and VMM maps GPA→HPA. Hardware two-stage EPT/NPT combines both during walking. A shadow table is a VMM-maintained GVA→HPA composition used by an ordinary MMU, requiring interception/tracking of guest table changes. Extra tables do not make them one algorithm.

Shadow tables and hardware two-stage translation (continued)

With4096 B pages, GVA0x1234=(VPN1, offset 0x234); guestPTE[1]=(GFN5, rw) gives GPA0x5234. EPT[5]=(MFN0xA, r) gives HPA0xA234. Read needs permission at both; EPT denies write despite guest w. Guest-table denial follows guest fault semantics; second-stage violation uses VMM handling. They are not one identical fault.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why uncached nested walks are not twice as expensive

Assume m guest levels, n second-stage levels, no TLB/walk caches or large pages. Each guest PTE read requires n second-stage references plus its own read, totaling m(n+1). Translating the final data GPA adds n. Thus

$$T_{tables} = m(n+1) + n = (m+1)(n+1) - 1.$$

Add one final data access. For m=n=4,24 table reads plus data=25, not4+4. This is a cold model, not every load. TLB hits, walk caches, huge pages lower it. Shadows have a shorter hardware walk but VMM maintenance on guest updates.

Invalidate translation caches under architectural rules on mapping/permission changes. Correct tagging or flushing must prevent one VM's equal GVA from reusing another's mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VM types and implementations

Compare lifecycle, type 0/1/2, PV, process VMs, emulation, and containment by abstraction. Shared-ring counters, ownership, and publication order preserve state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lifecycle and type responsibilities

Creation validates CPU, memory, disk, network needs and policy, allocates vCPUs, address spaces, devices, then loads the guest. Running VMs may pause, resume, snapshot, migrate. Deletion first stops devices/events before resource reclamation; deleting management metadata while DMA/network identity remains active is insufficient.

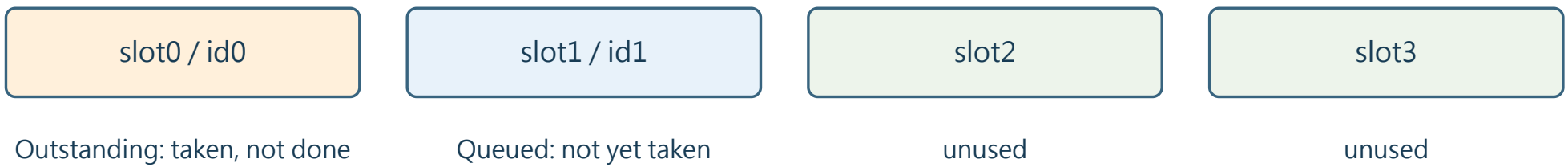
Type0 partitions via firmware/hardware with dedicated/shared resources. Type1 directly manages host resources, including VMMs in some general-purpose kernels. Type2 uses host process/device/memory services, often with privileged helpers/drivers and hardware assistance. Type2 does not mean no host awareness or no VT-x.

Types do not universally rank performance/functionality. Nesting depends on VMM, CPU, configuration, not solely type 0. Compare management scope, TCB, and resource paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Paravirtual I/O and a four-slot ring

Four slots, ordered completion: backend consumption does not free a slot



req_prod=2 · req_cons=1 · rsp_prod=0 · rsp_cons=0 ; used=2 · free=4−2=2 。

Complete id0: rsp_prod→1. Frontend consumes response, rsp_cons→1; only then release the slot.

Historical reference is Xen SOSP2003 cooperation, hypercalls, shared rings. This simplified single-producer/consumer model is not every Xen ABI. Counters increase monotonically; slot=index mod4. With req_prod2, req_cons1, rsp_prod0, rsp_cons0, queued=1, outstanding=1, response=0, used=2, free=2.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Paravirtual I/O and a four-slot ring (continued)

Backend completing id0 advances rsp_prod1, making outstanding0, response1. Frontend consumes it, advances rsp_cons1, leaving used=req_prod−rsp_cons=1, free3. Backend taking a request does not permit descriptor reuse while completion is pending.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ring synchronization and validation

Write descriptor before release-publishing req_prod. Consumer acquire-observes prod before reading descriptor. Write response before publishing rsp_prod; frontend consumes it before advancing rsp_cons to release the slot. Event-channel/doorbell notifications do not replace counter predicates. Recheck before sleeping against lost wakeups.

Maintain$0 \le \text{req_prod} - \text{rsp_cons} \le N$ and $\text{rsp_cons} \le \text{rsp_prod} \le \text{req_cons} \le \text{req_prod}$ in this ordered-completion model, keeping regions disjoint. Backend validates ID, operation, block bounds, length, and grants/ownership. Shared memory does not authorize arbitrary guest DMA into host. Out-of-order completion needs another ID/slot model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Process VMs, emulation, and containment

JVM/.NET expose bytecode/runtime rather than a full guest OS. Cross-ISA emulation interprets/translates instructions/machine state to run incompatible binaries. Neither is classified solely by owning hardware ring 0.

Solaris10 zones/BSD jails share the host kernel while separating namespaces, resources, and security contexts. Different userland/network views do not create independent kernels. Shared-kernel failure differs from VM isolation. Solaris global zone manages non-global zones; services/resource controls need configuration rather than an independent kernel scheduler per zone.

Testing another kernel needs a system VM/emulator; separating same-kernel services can use containment; bytecode needs its runtime. Choose an appropriate interface rather than a universal winner.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Virtualization and OS components

Guest schedules threads; host schedules vCPUs. CPU runtime differs from wall time. Ballooning, COW, IOMMU, storage, and migration require explicit states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two scheduling layers

Two pCPUs serve A 0/A 1 and B0/B1, ignoring host overhead. A runs0–10 ms, B10–20, A 20–30, B30–40. A thread needing20 ms CPU on A 0 completes at 30; B's counterpart at 40. Equal long-run halves do not make every20 ms budget have40 ms response.

Guest-ready threads wait while their vCPU lacks pCPU time. Descheduling a guest lock holder can waste another vCPU's spinning allocation. Separate virtual clock, wall clock, vCPU runtime. Expired virtual timers may await vCPU scheduling. Guest priorities alone cannot provide host-level real-time admission/isolation/scheduling guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Balloon direction and capacity conservation

Use Waldspurger's ESX OSDI2002 design. A guest balloon driver allocates/reserves guest pages and reports PFNs; VMM can reclaim their host backing while guest policy chooses other pages to reclaim/swap. Inflation reduces workload-usable guest memory; it creates no RAM.

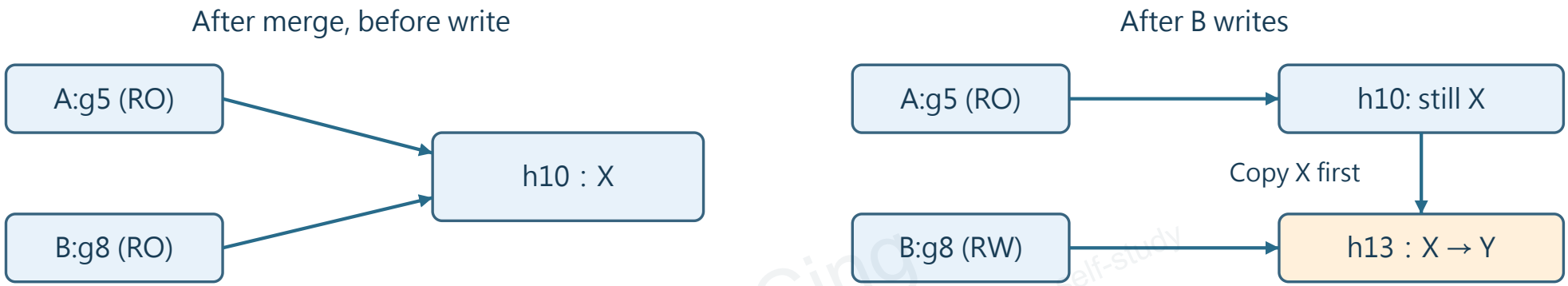
A 4096 MiB host reserves512, leaving3584 guest backing. A/B demand2304 each,4608 total not yet fully backed, exceeding availability by1024. Balloon A 256 and B768 to target A 2048+B1536=3584, plus VMM 512=4096. This conserves demand/allocation targets; it does not claim physical RAM once held4608 guest MiB simultaneously. Nominal sizes can remain while usable backing changes at a cost.

Balloon pages cannot remain workload-accessible. Protect unbacked mappings; erroneous guest access must fault/reback rather than reach frames reassigned elsewhere. Deflation restores safe backing first. Uncooperative/slow drivers can require VMM paging/limits; balloon success is not unconditional.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sharing, COW, and double paging

Compare bytes before sharing; split host frames before writing



B write fault: allocate/copy → change B mapping and invalidate → allow write. A stays unchanged.

A:g5→h10 and B:g8→h12 both contain X. Hashes select candidates; full-byte comparison and synchronization precede merging into h10 with write removed and references tracked. B's write faults: copy X to h13, change B:g8→h13, invalidate old translation as required, then allow Y. A retains X. Never allow write before splitting.

Sharing, COW, and double paging (continued)

VMM swapping lacks guest victim knowledge. Guest can write to virtual swap while VMM also swaps the host frame holding its page, causing double paging/extra I/O. Ballooning lets guest choose victims but is not universally faster. Sharing scans cost CPU/synchronization and depend on tenant-isolation policy; cross-tenant sharing is not assumed always enabled.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I/O paths and virtual networking

Emulation validates virtual-register operations before host-driver service. Paravirtual devices use cooperative queues. Passthrough lets a device serve a guest directly, still requiring IOMMU isolation, interrupt routing, and safe reset/ownership transfer.

If DMA0x5000 maps to host 0xA000, IOMMU must confine the permitted range. CPU EPT alone is not DMA confinement. Passthrough may require extra device-state support for snapshots/migration; RAM copying alone is insufficient.

Bridging joins an L2 network. NAT maps internal addresses/ports to external tuples. A 10.0.2.15:5000 and B10.0.2.16:5000 can use host:40001/40002 and share one public IP. Firewalls still define guest-to-guest/external channels.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Storage mappings and snapshot chains

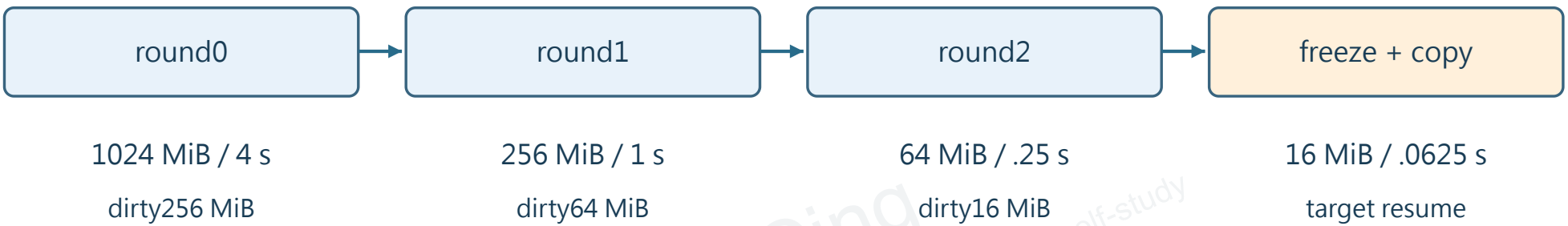
Guest LBA maps through an image/host filesystem to storage, or directly to a block volume; a virtual disk need not be one ordinary file. Thin provisioning postpones allocation without guaranteeing all logical capacities can be filled together.

Base={0:A,1:B,2:C}; after snapshot, active delta={1:B′}. Read1 finds B′ in delta, read 2 falls through to C. Selecting base restores B at 1 but does not retract sent mail. A shared base cannot be deleted while clones reference it. Backups need the required chain, metadata, and consistent point.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pre-copy migration round by round

Normal pre-copy: total time differs from downtime



B=256 MiB/s, d=64 MiB/s. Total 1360 MiB / 5.3125 s; pause .0625 s, excluding control cost.

Assume compatible VMM/CPU, shared storage, no unmigratable device, effective copy B=256 MiB/s, distinct dirtied pages d=64 MiB/s, and ignore protocol/CPU-state cost. R0=1024 copies in 4 s, creating R1=256; next1 s creates R2=64; next0.25 s creates R3=16. Freeze and copy16 in 0.0625 s. Total1360 MiB over5.3125 s; pause0.0625 s.

Pre-copy migration round by round (continued)

Generally $R(i+1)=(d/B)R(i)$, geometrically shrinking if $d/B<1$. At $d \geq B$ it does not shrink; cap rounds, reduce dirtying, extend pause, or change method. Repeated dirty pages, variable bandwidth, and finite RAM make reality richer than this explicit model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stop-and-copy and one active owner

Freeze includes state-changing DMA/devices/events, not only user code. Save vCPU/device/dirty RAM and required network state. Source must stop externally executing the guest before ownership transfer; reclaim source after target operation is established.

If target starts but acknowledgment is lost, source restarting can create split brain. A safe protocol needs explicit commit/ownership and failure recovery. Waiting for acknowledgment alone is not a proof of all failure cases. This simplified normal path proves a consistent handoff, not transparent availability under arbitrary partitions.

Maintain external address/MAC/routing reachability. TCP tolerating a short pause does not make downtime zero. Shared storage is this model's premise; block migration exists elsewhere, so it does not prove all live migration excludes disk copying.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Examples

Compare the book's Workstation architecture with Xen2003/ESX2002 research. Java SE 7 bytecode/runtime is a different abstraction from system VMs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Workstation, Xen, and ESX

The book's circa2013 hosted Workstation diagram routes guest app→guest kernel→virtual device/CPU→VMM/host services→hardware. Ordinary instructions can execute directly; I/O can use host services. It is an architecture example, not a release's complete implementation or measurement.

Xen SOSP2003 uses cooperative guests/hypercalls/shared rings. ESX OSDI2002 manages overcommit with ballooning/content sharing. Their periods/interfaces differ. Early Xen PV is not every later Xen mode; one paper benchmark does not establish a universal performance ratio.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

JVM verification and execution

In a Java SE 7-era abstraction, class loading/verification checks types, operand stacks, and legal control flow. Execution interprets or JIT-compiles; runtime supplies heap/GC/security. JVM has no separate general-purpose guest kernel and still uses host memory, threads, I/O.

For iconst_2;iconst_3;iadd;istore_1, stack is[]→[2]→[2,3]→[5]→[], local1=5. Starting empty with iadd lacks its two integers and can be rejected by verification before a hardware underflow. This does not prove business correctness.

JIT can use hotness/tiering rather than compile a whole method on first call. GC follows reachability/collector rules rather than deciding semantic future usefulness. Separate language specifications from implementation choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two required properties

Fidelity preserves specified guest-visible transitions; resource control prevents escaping CPU, address, device management. Traps, translation, hypercalls, hardware assistance implement these properties rather than prove themselves.

Permission intersection, ring publication/reuse, COW-before-write, and single-owner migration keep guest-visible and host-managed state consistent. A successful demonstration does not cover every transition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Denominators and boundaries

VCPU count is not pCPU capacity; nominal RAM is not current host backing; logical disk size is not allocated bytes; snapshots do not roll back the outside world; migration transfer time is not downtime. Identify the layer of every number. Exercises reconstruct modes, addresses, walk counts, ring counters, balloon capacity, COW, and migration recurrences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Silberschatz, Galvin, and Gagne,*Operating System Concepts*,9th edition, Chapter 16, printed pages711–738. Figures/cases are original; shadow and hardware two-stage translation remain separate.

Xen, SOSP2003, § 2–3 gives historical PV/hypercall/shared-I/O context. ESX ballooning and content sharing, OSDI2002 cover guest allocation/host reclamation, byte comparison, and COW. JVM SE 7 § 4.10 specifies verification types/stack rules. Capacity/migration calculations use explicit original assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study